

# SC2006 Software Engineering

## Lecture 13 Supplement

Conrad Watt

## Lt12 brief recap

- Architectural styles are a lightweight but high impact way to begin to organise the details of your system
- Focused on arranging the top-level components of the system from an implementer's point of view, as opposed to a user's point of view (contrast - use case diagrams)
- Generally made up of two fundamental pieces
  - components
    - boxes representing sub-parts of the system
    - sub-architectures can be nested inside components
  - connectors:
    - lines representing interaction between components

## Lt12 brief recap

- You saw the layered architecture
  - Three layer architecture maps well onto previous course-half's "stereotype diagrams"
- You were encouraged to think about abstraction more broadly
  - architectures abstract the underlying system design
  - in this lecture we will talk about design patterns, which are useful implementation abstractions at a more fine-grained/object level

# Design patterns

- Design patterns were popularised in 1994 by Erich Gamma, Richard Helm, Ralph Johnson and John Vlissides in their book *Design Patterns*
- Sometimes referred to as the “Gang of Four” (no relation)



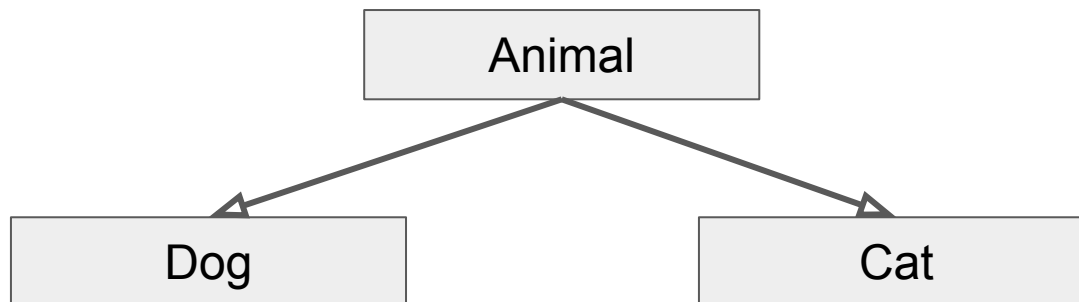
# Design patterns



- Peter Norvig classified the presence and use of design patterns into three categories
- **Invisible**
  - “so much a part of language that you don’t notice”
  - examples would include Java classes for the Strategy pattern
- **Informal**
  - Implemented in code from scratch, as needed for an individual problem
- **Formal**
  - Implemented as a library, specialised for a particular use when needed
  - example: the `oracle.javatools.patterns` library for Java

# Design patterns

- Several popular patterns are used **invisibly** in certain languages
- e.g. the Strategy pattern in Java



# Design patterns

- Several popular patterns are used **invisibly** in certain languages
- e.g. the Observer pattern in JavaScript

```
button.addEventListener("click", observerObj)
```

# Design patterns

- A common criticism of design patterns is that they promote inflexible thinking about code

“When I see patterns in my programs, I consider it a sign of trouble. The shape of a program should reflect only the problem it needs to solve. Any other regularity in the code is a sign, to me at least, that I'm using **abstractions** that aren't powerful enough...”

- **Paul Graham**





# Design patterns

- Generally speaking, a conscious choice to make repeated **informal** use (in the Norvig sense) of a design pattern attracts the most criticism
- Some design patterns have survived in common parlance, in particular
  - **observer pattern**
  - visitor pattern
  - **factory pattern**
  - builder pattern
  - decorator pattern

# Design patterns

- Other design patterns are not commonly referenced explicitly by name, but this doesn't mean they aren't used
- e.g. strategy pattern is baked into many modern languages
- a broad trend towards *asynchronous* programming language features (like JavaScript promises, Rust futures) could be seen as widescale adoption of the observer pattern